# DAYANANDA SAGAR ACADEMY OF TECHNOLOGY & MANAGEMENT

(An Autonomous Institute under VTU)

Department of CSE-Cyber Security

7th Semester 2026-27

**Project Phase 2- First Review**

## SHADOW-CODE: AGENTIC AI FRAMEWORK FOR AUTONOMOUS SECURE SDLC AUDITING


| | | |
|---|---|---|
| 1DT23CY002 | : | ABHINAV RAJ |
| 1DT23CY003 | : | ABHISHEK ANAND |
| 1DT23CY022 | : | IKSHIT SINHA |
| 1DT23CY040 | : | SACHIN SHEKHAR |

**GUIDE NAME: Mr. Anitha R ,**

**ASSOCIATE PROFESSOR**

# Index


- Abstract
- Problem Statement
- Objectives
- Literature Survey
- Motivation
- Proposed Contribution
- Architectural Flow
- Architectural Design
- Implementation Status
- Conclusion

# Abstract


- **Problem Statement:** Modern CI/CD pipelines have increased the speed of software development while also increasing the risk of introducing vulnerabilities.

- **Security Gap:** Traditional SAST approaches mainly depend on predefined, rule-based pattern matching and may struggle with semantic intent and complex business-logic vulnerabilities.

- **Solution:** Shadow Code proposes an autonomous, multi-agent DevSecOps security layer that intercepts code commits and performs security analysis before changes reach the main repository.

- **Proposed Framework:** Shadow Code uses 8 specialized AI agents coordinated through an LLM-based workflow for code analysis, vulnerability detection, CVE mapping and remediation.

- **Adaptive Intelligence:** A human-in-the-loop feedback mechanism is proposed to incorporate developer interactions and improve repository-specific security analysis.

# Problem Statement

## Existing Problem

- Modern CI/CD pipelines prioritize rapid development and deployment.
- Security auditing needs to keep pace with frequent code changes.
- Traditional SAST tools primarily depend on predefined security rules.
- Rule-based analysis may miss semantic and business-logic vulnerabilities.
- False positives can result in developer alert fatigue.
- Existing approaches often emphasize vulnerability detection rather than complete detection-to-remediation workflows.

## Problem Addressed

- Develop an intelligent security layer capable of analyzing source code beyond simple pattern matching and assisting developers in identifying and remediating vulnerabilities before code reaches the main repository.

# Objectives

1. **Shift Security Left**
   Identify security vulnerabilities as early as possible during software development rather than waiting until deployment.

2. **Continuous Security Auditing**
   Integrate security analysis into the developer workflow and CI/CD pipeline.

3. **Semantic Code Analysis**
   Use LLM-based reasoning to analyze the intent, context and logic of source code.

4. **Automated Remediation**
   Provide context-aware recommendations and generate potential secure fixes for identified vulnerabilities.

5. **Adaptive Intelligence**
   Use developer feedback, such as accepted or overridden findings, to improve future security analysis.

6. **Vulnerability Intelligence**
   Correlate detected findings with known vulnerabilities and CVE information to provide additional context.

# Literature Survey

| Paper Title | Author | Year | Gaps Identified (Addressed by Shadow Code) | Techniques Used |
|---|---|---|---|---|
| **LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights** | Z. Sheng, et al. (arXiv) | 2025 | Struggles with high false-positive rates; lacks a multi-agent orchestration system directly integrated into the developer's Git push pipeline. | Single-agent LLM analysis (GPT, CodeBERT), Static/ Dynamic analysis integration. |
| **From Large to Mammoth: A Comparative Evaluation of LLMs in Vulnerability Detection** | NDSS Symposium | 2025 | Focuses solely on detection rather than remediation; performance fluctuates heavily without traditional static tools (like Semgrep) acting as a pre-filtering layer to save tokens. | File-level vulnerability classification using open-source models (LLaMA-3, Mistral, Gemma). |
| **Detecting Code Vulnerabilities using LLMs** | IEEE DSN | 2025 | Patching success rate is limited because it lacks an adaptive feedback loop (like Agent 8) that learns from human developer overrides and team-specific coding styles. | Dynamic prompt engineering, Zero-shot Chain-of-Thought (CoT), LLM-based vulnerability patching. |
| **LLM-Based Multi-Agent Systems for Software Engineering: Literature Review and Vision** | X. Yin, et al. (arXiv) | 2024 | Highly theoretical; lacks an end-to-end functional architecture that handles everything from initial code parsing (tree-sitter) to automated CVE mapping via Vector Databases (ChromaDB). | Multi-agent collaboration frameworks, role-playing agents (Tester vs. Developer), Consensus-based classification. |

# Motivation

## Why Shadow Code?

**1. Increasing Development Speed**

Rapid CI/CD workflows require security analysis to happen continuously and efficiently.

**2. Limitations of Traditional SAST**

Rule-based scanners are effective for known patterns but may have difficulty understanding complex code context and business logic.

**3. False-Positive Problem**

Excessive warnings can lead to developer alert fatigue and cause important vulnerabilities to be overlooked.

**4. Need for Context-Aware Analysis**

LLMs can complement traditional scanners by reasoning about the purpose and behavior of source code.

**5. Early Security Intervention**

Detecting and addressing vulnerabilities during development can reduce the possibility of insecure code reaching the main repository.

# Proposed Contribution

**1. Real-Time Threat Intelligence**

Integrate vulnerability information from the NVD/CVE ecosystem to provide context for detected security issues.

**2. Intelligent Pre-Filtering**

Use deterministic scanners such as Semgrep, Bandit or ESLint before LLM analysis to identify obvious security issues and reduce unnecessary LLM processing.

**3. Semantic Vulnerability Analysis**

Use an LLM-based agent to reason about code intent and identify vulnerabilities that may not be easily captured by predefined patterns.

**4. Fail-Safe Remediation**

Generate potential fixes and validate the modified code before considering the remediation successful.

**5. Human-in-the-Loop Feedback**

Allow developers to accept, reject or override findings so that their decisions can contribute to future analysis.

**6. Development Workflow Integration**

Position Shadow Code as a security gate between the developer's code changes and the remote repository.

# Architectural Flow:

**The Shadow Code workflow follows a sequential and reactive architecture orchestrated using LangGraph.**

**Stage 1 — Trigger & Interception**

**Developer → Git Push → Shadow Code Gateway**

- Developer initiates a Git push.
- Shadow Code Gateway intercepts the change.
- Code enters a temporary quarantine state.
- LangGraph activates the security-analysis pipeline.

**Stage 2 — Structural Analysis**

**Parser → Static Scanner**

- Tree-sitter generates an Abstract Syntax Tree (AST).
- Source code is divided into logical units such as functions/classes.
- Non-code files can be filtered.
- Static tools perform initial security checks.

**Stage 3 — Semantic Analysis**

**AI Semantic Reasoner**

- LLM analyzes the context and intent of the code.
- It looks for deeper or context-dependent vulnerabilities.

# Architectural flow:-

**Stage 4 — Threat Intelligence**

**CVE Correlator**

- Detected findings are compared with vulnerability intelligence.
- NVD/CVE information provides additional context.

**Stage 5 — Ranking & Remediation**

**Severity Ranker → Report Writer → Surgeon**

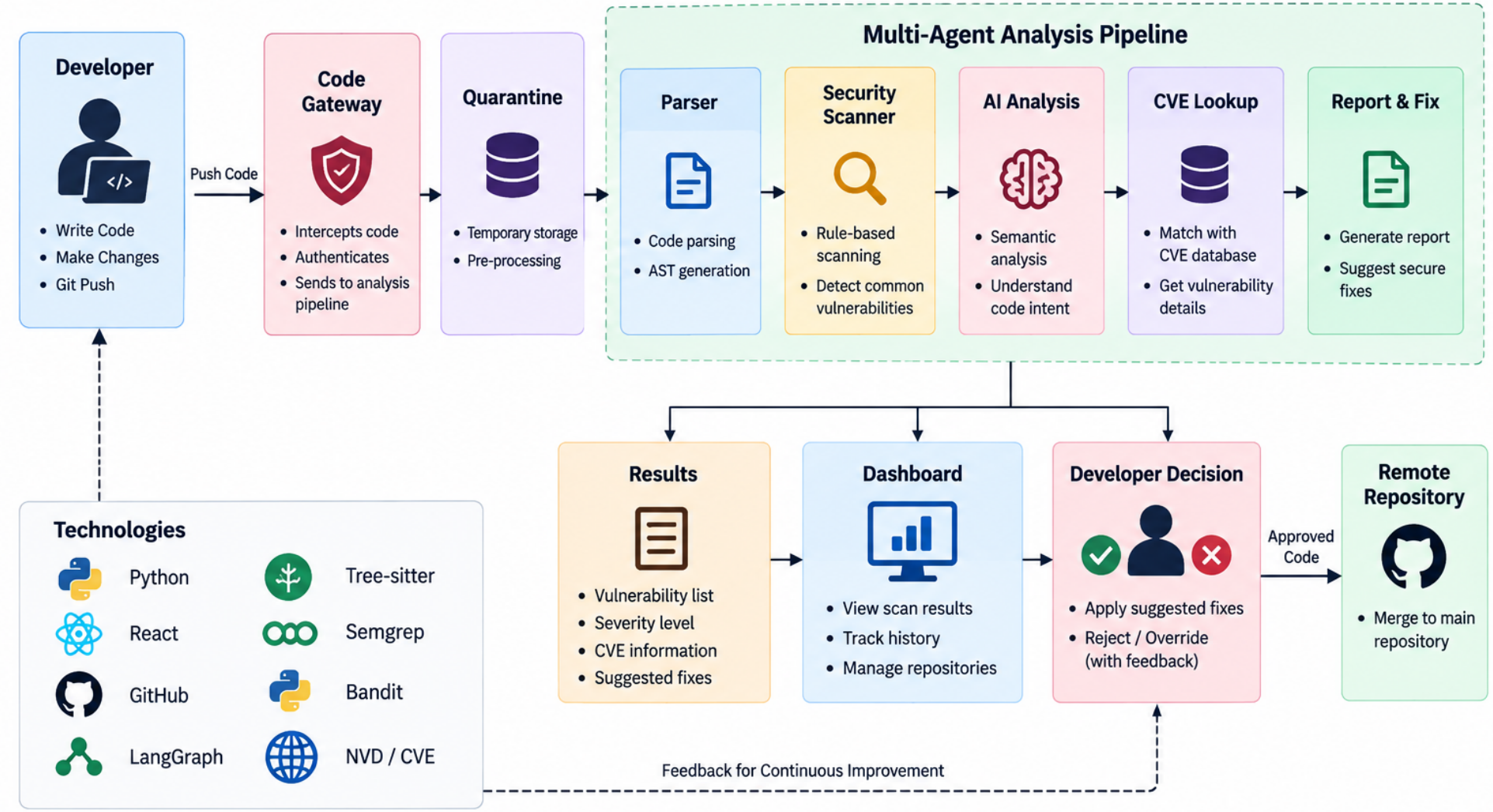
- Findings are ranked by severity.
- Security reports are generated.
- Potential fixes are generated and validated.

**Stage 6 — Developer Feedback**

**Developer Decision → Adaptive Memory**

- Developer accepts or overrides findings.
- Feedback can be used to improve future analysis.

# Architectural Design:-



# Implementation status

## Completed Work

**Frontend & Dashboard**

- Designed the UI/UX for the Shadow Code dashboard.
- Developed the frontend using React.
- Implemented a GitHub push tracking interface.
- Implemented code-analysis result display.
- Added vulnerability status indicators:
  - Low
  - Medium
  - High

**Dashboard Features**

- GitHub push activity
- Analysis reports
- Scan logs
- Scan history
- Vulnerability status visualization

## Current Development

- AI-based vulnerability insights display.
- Further integration of AI analysis into the workflow.

# THANKYOU